

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO



**CIENCIAS DE LA COMPUTACIÓN
INGENIERÍA EN SOFTWARE
ECUADOR**

INGENIERÍA DE REQUERIMIENTOS

**COMPARACIÓN ENTRE MODELOS DE PROCESO DE REQUISITOS EN
ESCENARIOS REALES**

AUTORES:

**CEDEÑO AVILA WINSTON DAMIAN
CORDOVA CARREÑO MAYRA LUCILA
MENDOZA PARRAGA ANDY JOHEL**

GRUPO: B

DOCENTE: ING. GUERRERO ULLOA GLEISTON CICERON

CURSO/PARALELO: CUARTO SOFTWARE “B”

PRÁCTICA EN CLASE: COMPARACIÓN ENTRE MODELOS DE PROCESO DE REQUISITOS EN ESCENARIOS REALES

Criterio	Cascada	Iterativo	Ágil
Descripción del modelo	El modelo de requisitos en cascada sigue un proceso lineal y secuencial, donde cada etapa se completa antes de pasar a la siguiente, iniciando con la definición de requisitos y continuando con diseño, implementación, pruebas y mantenimiento [1], [2].	El modelo iterativo se desarrolla en ciclos sucesivos donde los requisitos evolucionan con el tiempo, definiendo los requisitos más importantes en las etapas iniciales [3].	El modelo Ágil gestiona los requisitos de forma iterativa, permitiendo cambios y mejoras continuas según las necesidades del usuario durante el desarrollo. Prioriza la adaptación constante antes que seguir un plan rígido [4].
Interacción con el cliente	Ocurre principalmente al inicio del proyecto, durante la recopilación de requisitos y al final en la entrega del sistema. La participación continua del usuario es limitada [1], [5].	El cliente es contactado al inicio de cada iteración para definir requisitos y al final para verificar que el producto los cumple [6].	Mantiene una interacción continua con el cliente durante todo el desarrollo del proyecto. El usuario participa en reuniones de revisión, validación de funcionalidades y priorización de requisitos [7].
Frecuencia de entregas	El producto generalmente se entrega una sola vez al finalizar todas las etapas del desarrollo [8].	Se entregan de forma progresiva al final de cada iteración, ofreciendo versiones funcionales tempranas para evaluar los requisitos, aunque esto podría postergar la entrega final [3], [6].	Las entregas se realizan de forma frecuente y continua, generalmente cada 1 a 4 semanas. Cada sprint produce una versión funcional del sistema que puede ser evaluada por el cliente [9].
Capacidad de adaptación a cambios	Tiene poca flexibilidad porque los requisitos quedan definidos desde el inicio y los cambios posteriores son difíciles de implementar [2], [10].	Permite incorporar cambios o requisitos faltantes en la siguiente iteración, adaptando el desarrollo a nuevos objetivos sin interrumpir el ciclo actual [11].	Posee una alta capacidad de adaptación debido a su estructura flexible. Los cambios pueden incorporarse incluso en etapas avanzadas del desarrollo. El backlog y la planificación iterativa permiten reorganizar prioridades rápidamente [4].
Tiempo de respuesta ante cambios	Es lento debido a que cualquier modificación puede requerir regresar a fases anteriores y rehacer documentación y procesos [5], [12].	Responde rápidamente a los cambios, ya que los requisitos pueden ajustarse y refinarse en cada iteración [11], [13].	Gracias a los sprints cortos y la retroalimentación continua, permite responder rápidamente a cambios y nuevos requisitos [9].
Limitaciones del modelo	Presenta rigidez, poca adaptación a cambios, detección tardía de errores, escasa retroalimentación continua y dificultad para proyectos complejos o dinámicos [1], [8], [12].	Puede aumentar los costos y el tiempo de desarrollo debido a las múltiples iteraciones y cambios frecuentes en los requisitos [13].	Depende de la comunicación continua y del compromiso del cliente. Además, la flexibilidad de requisitos puede dificultar la estimación exacta de costos y tiempos [14].
Proceso de validación y pruebas	Las pruebas se realizan principalmente después de la implementación del sistema. Primero se aplican pruebas unitarias y luego pruebas integrales para verificar el cumplimiento de los requisitos [5], [8].	Los requisitos se validan y prueban continuamente en cada iteración mediante revisiones permitiendo detectar errores y realizar mejoras progresivas al sistema [15].	Incorpora pruebas continuas durante el desarrollo. Cada sprint incluye validación, integración y revisión del software mediante pruebas unitarias, funcionales y retroalimentación del cliente [7].

Escenarios reales de aplicación	Se aplica en proyectos con requisitos claros, estables y bien documentados, especialmente en proyectos pequeños, sistemas empresariales tradicionales o entornos donde los cambios son mínimos [1], [10].	Se aplica en proyectos de software complejos y colaborativos, en los que los requisitos evolucionan continuamente durante el desarrollo [11].	Se recomienda para proyectos dinámicos y con cambios frecuentes en los requisitos, como desarrollo web, aplicaciones móviles, startups y plataformas digitales centradas en el usuario [16].
--	---	---	--

REFERENCIAS BIBLIOGRÁFICAS

- [1] S. Pargaonkar, “A Comprehensive Research Analysis of Software Development Life Cycle (SDLC) Agile & Waterfall Model Advantages, Disadvantages, and Application Suitability in Software Quality Engineering,” *International Journal of Scientific and Research Publications*, vol. 13, no. 8, pp. 120–124, Aug. 2023, doi: 10.29322/IJSRP.13.08.2023.p14015.
- [2] H. Prabowo, F. L. Gaol, and A. N. Hidayanto, “Comparison of the System Development Life Cycle and Prototype Model for Software Engineering,” *International Journal of Emerging Technology and Advanced Engineering*, vol. 12, no. 4, pp. 155–162, Apr. 2022, doi: 10.46338/ijetae0422_19.
- [3] I. Alam, N. Sarwar, and I. Noreen, “Statistical analysis of software development models by six-pointed star framework,” *PLoS One*, vol. 17, no. 4, p. e0264420, Apr. 2022, doi: 10.1371/journal.pone.0264420.
- [4] M. Kuhrmann *et al.*, “What Makes Agile Software Development Agile?,” *IEEE Transactions on Software Engineering*, vol. 48, no. 9, pp. 3523–3539, Sep. 2022, doi: 10.1109/TSE.2021.3099532.
- [5] N. S. Wisidagama and F. Marikkar, “WATERFALL MODEL OVER PCD.UCT MODEL REVIEW,” *Automation of technological and business processes*, vol. 16, no. 3, pp. 126–131, Oct. 2024, doi: 10.15673/atbp.v16i3.2927.
- [6] E. M. T. Sanmocte and J. A. Costales, “Exploring Effectiveness in Software Development: A Comparative Review of System Analysis and Design Methodologies,” *International Journal of Computer Theory and Engineering*, vol. 17, no. 1, pp. 36–43, 2025, doi: 10.7763/IJCTE.2025.V17.1367.
- [7] D. Strode, T. Dingsøyr, and Y. Lindsjorn, “A teamwork effectiveness model for agile software development,” *Empir. Softw. Eng.*, vol. 27, no. 2, p. 56, Mar. 2022, doi: 10.1007/s10664-021-10115-0.
- [8] A. Saravanos and M. X. Curinga, “Simulating the Software Development Lifecycle: The Waterfall Model,” *Applied System Innovation*, vol. 6, no. 6, p. 108, Nov. 2023, doi: 10.3390/asi6060108.
- [9] A. Anand, J. Kaur, O. Singh, and O. H. Alhazmi, “Optimal Sprint Length Determination for Agile-Based Software Development,” *Computers, Materials & Continua*, vol. 68, no. 3, pp. 3693–3712, 2021, doi: 10.32604/cmc.2021.017461.

- [10] L. Yuge and T. Badarch, “Research on Contemporary Software Development Life Cycle Models,” *American Journal of Computer Science and Technology*, Mar. 2023, doi: 10.11648/j.ajcst.20230601.11.
- [11] A. Sadovykh, B. Said, D. Truscan, and H. Bruneliere, “An iterative approach for model-based requirements engineering in large collaborative projects: A detailed experience report,” *Sci. Comput. Program.*, vol. 232, p. 103047, 2024, doi: <https://doi.org/10.1016/j.scico.2023.103047>.
- [12] A. Mishra and Y. I. Alzoubi, “Structured software development versus agile software development: a comparative analysis,” *International Journal of System Assurance Engineering and Management*, vol. 14, no. 4, pp. 1504–1522, Aug. 2023, doi: 10.1007/s13198-023-01958-5.
- [13] O. J. Okesola, A. A. Adebisi, A. A. Owoade, O. Adeaga, O. Adeyemi, and I. Odun-Ayo, “Software Requirement in Iterative SDLC Model,” 2020, pp. 26–34. doi: 10.1007/978-3-030-51965-0_2.
- [14] M. Wiesche, “Interruptions in Agile Software Development Teams,” *Project Management Journal*, vol. 52, no. 2, pp. 210–222, Apr. 2021, doi: 10.1177/8756972821991365.
- [15] M. I. H. -, “Software Development Life Cycle (SDLC) Methodologies for Information Systems Project Management,” *International Journal For Multidisciplinary Research*, vol. 5, no. 5, Sep. 2023, doi: 10.36948/ijfmr.2023.v05i05.6223.
- [16] M. Zorzetti, I. Signoretti, L. Salerno, S. Marczak, and R. Bastos, “Improving Agile Software Development using User-Centered Design and Lean Startup,” *Inf. Softw. Technol.*, vol. 141, p. 106718, Jan. 2022, doi: 10.1016/j.infsof.2021.106718.

ANEXO

Link repositorio en GitHub: <https://github.com/andymendozap03/IngReq-GrupoB-4toB.git>